

Highlights

Repair-Guided Evolutionary Optimisation (CREO): A Structured Search Paradigm for Continuous Optimisation

Alan Immanuel Benjamin Vallavaraj

- Repair-induced displacement is reused as an evolutionary search direction.
- A matched ablation isolates the effect of the repair-guidance term.
- Fixed repair guidance helps selected low-dimensional benchmark functions.
- Performance deteriorates when dimensionality increases from 10 to 30.

Repair-Guided Evolutionary Optimisation (CREO): A Structured Search Paradigm for Continuous Optimisation

Alan Immanuel Benjamin Vallavaraj^{a,*}

^a*School of Computer Science and Engineering, University of Westminster, London, United Kingdom*

ARTICLE INFO

Keywords:

evolutionary computation
differential evolution
repair operators
constraint handling
continuous optimisation
CEC2017 benchmark

ABSTRACT

Population-based metaheuristics are widely used for nonlinear, multimodal, and derivative-free optimisation. In conventional evolutionary optimisation, repair operators are normally used to restore feasibility or enforce search-domain bounds, after which the displacement induced by repair is discarded. This paper introduces Repair-Guided Evolutionary Optimisation (CREO), a structured-search framework that retains the repair displacement vector and reuses it as directional information during evolutionary updates. A Differential Evolution instantiation, CREO-DE, combines differential mutation, attraction toward the best repaired candidate, repair-guided displacement, and stochastic exploration.

CREO-DE is evaluated on 29 functions from the CEC2017 real-parameter benchmark suite (F2 excluded), using 30 stochastic runs, dimensions $D = 10$ and $D = 30$, a population size of 50, and a common budget of $10,000D$ objective evaluations. The experiments apply an additional author-defined stability-preference and feasibility-handling protocol on top of the CEC2017 objective functions. The design separates a five-method benchmark comparison using different feasibility-handling modes from a matched ablation that disables only the repair-guidance term. Within that experimental protocol, Differential Evolution obtains the lowest aggregate rank at both dimensions. CREO-DE is nevertheless competitive on selected functions at $D = 10$, while its aggregate position deteriorates at $D = 30$. The ablation produces win/tie/loss counts of 12/0/17 at $D = 10$ and 1/0/28 at $D = 30$ against the non-guided variant.

The contribution is therefore mechanistic rather than a claim of universal performance superiority: CREO treats repair-induced displacement as an explicit optimisation signal and exposes the dimensional conditions under which that signal is useful or detrimental. The results motivate dimension-adaptive repair weighting, sensitivity analysis, and evaluation on explicitly constrained optimisation problems.

1. Introduction

Continuous optimisation problems arise throughout machine learning, control, signal processing, engineering design, and computational intelligence. Many such problems are nonlinear, multimodal, expensive to evaluate, or lack reliable gradient information, making derivative-free and population-based search attractive [1–3]. Metaheuristics do not guarantee uniform superiority across problem classes, and the no-free-lunch result provides a general reminder that performance advantages are necessarily problem-dependent [4]. Consequently, a new optimiser should be evaluated not only by its best cases but also by the conditions under which its mechanism helps or harms search.

Population-based optimisation has developed through several complementary traditions. Genetic Algorithms (GA) established evolutionary search through selection and recombination [5]; Particle Swarm Optimisation (PSO) introduced collective motion driven by individual and social information [6]; Differential Evolution (DE) introduced vector-difference mutation for real-valued search [7]; and Grey Wolf Optimiser (GWO) is a later population-based method based on hierarchical guidance [8]. Although their update rules differ, all of these methods must handle candidate vectors that violate bounds or other feasibility requirements.

Constraint handling has therefore become an important part of evolutionary optimisation. Widely used approaches include penalty functions, feasibility rules, stochastic ranking, decoder-based mappings, and

*Corresponding author.

✉ a.vallavaraj@westminster.ac.uk (A.I.B. Vallavaraj)

repair [9–12, 32, 33]. In a conventional repair pipeline, an infeasible candidate is transformed into an admissible candidate and the repaired point is evaluated. Depending on the algorithm, the transformation that produced it may be treated as a corrective side effect rather than retained as a separate search signal.

CREO is motivated by a different question: *can the displacement generated by repair be reused as search information?* If a raw candidate $\mathbf{x}$ is mapped to a repaired candidate $\mathcal{R}(\mathbf{x})$, then

$$\Delta\mathbf{x} = \mathcal{R}(\mathbf{x}) - \mathbf{x} \quad (1)$$

contains a direction and magnitude associated with movement toward the repair-defined admissible region. CREO retains this vector and injects it into subsequent evolutionary updates.

This paper develops that idea as a general structured-search framework and studies a DE-based instantiation. The work is deliberately framed as a mechanism study rather than a claim that CREO-DE is a universally stronger optimiser. The main contributions are:

- a formulation of repair displacement as an explicit search signal rather than a discarded by-product of repair;
- a generic CREO update rule and a Differential Evolution instantiation, CREO-DE;
- a CEC2017-derived evaluation with an explicitly documented auxiliary stability region, 29 objective functions, two dimensions, 30 runs per function, and a common objective-evaluation budget;
- an aggregate rank analysis and complete function-wise reporting for GA, PSO, DE, GWO, and CREO-DE; and
- a matched ablation that isolates the repair-guidance term and reveals strong dimensional sensitivity.

The remainder of the paper first positions CREO relative to evolutionary optimisation, swarm methods, hybridisation, and constraint handling. It then formalises the repair-guided mechanism, describes the benchmark and repository organisation, reports the two experiments, and discusses what can and cannot be concluded from the observed performance.

2. Related Work

2.1. Evolutionary Optimisation and Differential Evolution

Evolutionary optimisation encompasses population-based methods that iteratively generate, evaluate, select, and transform candidate solutions [15, 16]. GA remains one of the foundational paradigms [5], while later methods have specialised search operators for continuous decision spaces. DE is particularly relevant because it uses scaled differences between population vectors to create self-referential search steps [7]. Its simplicity, low number of control parameters, and strong performance on real-parameter benchmarks have motivated extensive research on adaptation and hybridisation [17].

Several influential DE variants focus on parameter control and strategy adaptation. Brest *et al.* proposed self-adapting control parameters [18]; Qin *et al.* developed strategy adaptation for global numerical optimisation [19]; and Tanabe and Fukunaga introduced linear population-size reduction in the SHADE family [20]. These developments show that the behaviour of DE can change substantially when additional information is used to control mutation, parameter schedules, or population structure. CREO-DE follows the same general philosophy of augmenting a strong baseline, but the added information source is the displacement created by repair.

Evolution Strategies provide another important line of continuous optimisation research. CMA-ES adapts a covariance matrix to learn the geometry of successful search steps [21]. CREO does not attempt to learn a full covariance model; instead, it asks whether a much cheaper signal already produced by the feasibility or boundary-handling process can be recycled into directional guidance.

2.2. Swarm and Nature-Inspired Search

PSO models information exchange through personal and global best positions [6]. Subsequent analyses investigated stability and convergence properties [22] and practical developments in swarm search [23]. Comprehensive Learning PSO extends the information-sharing mechanism by allowing dimensions to learn from different exemplars [24]. These approaches are relevant to CREO because they demonstrate that the *source and structure* of directional information can be as important as the random variation mechanism itself.

Other population-based methods include Ant Colony System [25], Artificial Bee Colony ideas [26], Firefly Algorithm [27], Whale Optimisation Algorithm [28], GWO [8], and Harris Hawks Optimisation [29]. The diversity of these methods has motivated broader comparisons and taxonomies of metaheuristics [14]. CREO is not proposed as another behavioural metaphor. Its defining element is an operator-level mechanism: information produced by repair is retained and reused in the search update.

2.3. Hybridisation and Multi-Operator Search

Hybrid methods combine complementary search processes to alter exploration and exploitation. Memetic algorithms formalised the integration of evolutionary search with problem-specific improvement mechanisms [31], while multiobjective evolutionary methods such as NSGA-II demonstrate how selection and dominance structures can be engineered around multiple criteria [30]. In the present work, CREO is best understood as an augmenting mechanism that can be embedded in another optimiser. CREO-DE is therefore a hybrid instantiation rather than an attempt to replace the underlying DE machinery.

2.4. Constraint Handling and Repair

Constraint handling in evolutionary computation has been studied for decades. Michalewicz and Schoenauer surveyed early constrained parameter-optimisation approaches and benchmark practices [32]. Coello provided a broad survey of penalty, repair, special representation, and hybrid techniques [9]; Deb proposed feasibility rules that avoid explicit penalty coefficients [10]; and Runarsson and Yao introduced stochastic ranking [11]. Later work continued to organise the field and identify open problems in nature-inspired constrained optimisation [12].

Repair methods are especially attractive when a deterministic mapping can move a candidate into an admissible region. Decoder-like and mapping-based strategies, including homomorphous mappings, show that feasibility restoration can itself impose useful structure on the search space [33]. CREO extends this observation by treating the *repair transformation* as data. Rather than evaluating a repaired point and discarding how it was obtained, CREO stores the repair displacement and uses it as a vector signal in the next search step.

2.5. Benchmarking and Comparative Evaluation

Real-parameter optimisation algorithms are commonly evaluated on benchmark suites that contain heterogeneous objective landscapes. The CEC2017 suite is designed for systematic comparison across unimodal, multimodal, hybrid, and composition functions [13]. Because objective magnitudes differ across benchmark functions, aggregate arithmetic means of raw objective values can be misleading. Rank-based summaries and nonparametric comparisons are widely used in evolutionary computation for cross-problem evaluation [34, 35]. The present paper therefore bases its cross-function conclusions primarily on function-wise ranks and win/tie/loss summaries.

3. Repair-Guided Evolutionary Optimisation

3.1. Two Distinct Feasibility Regions

The CEC2017 test functions are box-bounded real-parameter optimisation problems. The experiment scripts add a *separate, author-defined stability preference* to the standard search domain. Keeping these notions distinct is essential to interpreting the reported numbers. The benchmark bounds are

$$\Omega = [-100, 100]^D, \quad \mathcal{B}(\mathbf{x})_j = \min(100, \max(-100, x_j)). \quad (2)$$

For a boundary-clipped vector $\mathbf{q} = \mathcal{B}(\mathbf{x})$, the experiment defines the auxiliary violation score

$$V(\mathbf{q}) = \sum_{j=1}^D [\max(0, -80 - q_j) + \max(0, q_j - 80)] + D \max \left(0, \frac{1}{D} \sum_{j=1}^D |q_j| - 60 \right). \quad (3)$$

Thus $V = 0$ identifies vectors lying in the preferred coordinate interval $[-80, 80]$ with mean absolute coordinate at most 60. This *additional* preference is not an intrinsic constraint of the official CEC2017 functions. The experimental protocol does not change their objective function, but it does change which points are evaluated and how candidates are selected.

3.2. Implemented Iterative Repair and Displacement

The released CEC2017 scripts implement $\mathcal{R}$ as follows. Initially let $\mathbf{q} = \mathcal{B}(\mathbf{x})$ and $\mathbf{y}^{(0)} = \mathbf{q}$. If $V(\mathbf{q}) \leq 10^{-12}$, return $\mathbf{q}$ immediately. Otherwise, for $k = 0, \dots, 9$, compute

$$\mathbf{y}^{(k+1)} = \mathcal{B}(0.8 \mathbf{y}^{(k)}), \quad (4)$$

and stop as soon as $V(\mathbf{y}^{(k+1)}) \leq 10^{-12}$. The code has a ten-iteration cap; if that cap is reached, it returns the last iterate. Under the stated $[-100, 100]^D$ clipping and the stated 0.8 shrink factor, however, every coordinate has magnitude at most $100(0.8)^3 = 51.2$ by the third shrink step. Consequently, $V = 0$ is reached in no more than three iterations in the reported setting. The returned point is denoted $\mathcal{R}(\mathbf{q})$. We distinguish the official benchmark box Ω from the narrower author-defined $V = 0$ region.

For target i , the implementation stores a box-clipped population member $\mathbf{q}_i$, an evaluation record for its repaired point $\mathbf{z}_i = \mathcal{R}(\mathbf{q}_i)$, and the displacement

$$\mathbf{d}_i = \mathbf{z}_i - \mathbf{q}_i. \quad (5)$$

The displacement in this study primarily reflects the auxiliary stability-preference repair, *not merely boundary clipping*. No claim is made that the displacement necessarily points in a direction of objective improvement.

3.3. Evaluation and Feasibility-Aware Selection

The experiment scripts supply two evaluation modes. In *penalty mode* (used for GA, PSO, DE and GWO), the official CEC2017 objective is evaluated at $\mathbf{q}$, with stored fitness and feasibility descriptors

$$o_p = f_{\text{CEC}}(\mathbf{q}), \quad \phi_p = o_p + 10^6 V(\mathbf{q}), \quad (6)$$

$$h_p = \mathbf{1}[V(\mathbf{q}) \leq 10^{-12}].$$

In *repair mode* (used for CREO-Repair, CREO-RG and CREO-DE), the point evaluated by the same CEC function is $\mathbf{z} = \mathcal{R}(\mathbf{q})$, with

$$o_r = f_{\text{CEC}}(\mathbf{z}), \quad \phi_r = o_r, \quad (7)$$

$$h_r = \mathbf{1}[V(\mathbf{z}) \leq 10^{-12}].$$

The implementation's candidate-comparison rule first prefers a feasible candidate, then compares fitness when both candidates are feasible; if both are infeasible, it prefers lower violation before comparing fitness. Thus penalty mode is *not* equivalent to ranking all candidates solely by the scalar penalised objective. The recorded **objective** column reports o_p or o_r , rather than ϕ_p or ϕ_r . Because the two modes evaluate different points and apply different repair/penalty protocols, their raw objective results do not establish performance superiority on an unmodified CEC2017 protocol. They describe the particular augmented protocol used in this study.

3.4. CREO and Its DE Instantiation

The generic CREO idea is to retain the displacement created by an explicit repair mapping and inject it into an evolutionary update. In the *implemented* CREO-DE hybrid, indices r_1, r_2, r_3 are distinct and different from target i . The mutant generated from the stored box-clipped population $\mathbf{q}$ is

$$\mathbf{v}_i = \mathbf{q}_{r_1} + F(\mathbf{q}_{r_2} - \mathbf{q}_{r_3}) + \alpha(\mathbf{z}_{best} - \mathbf{q}_i) + \beta \mathbf{d}_i + \gamma \boldsymbol{\epsilon}_i, \quad (8)$$

where $\boldsymbol{\epsilon}_i \sim \mathcal{N}(\mathbf{0}, I_D)$, $F = 0.7$, $\alpha = 0.18$, $\beta = 0.90$, $\gamma = 0.02$, and the crossover rate is $CR = 0.9$. The vector $\mathbf{z}_{best}$ is the best recorded repaired evaluation point under the feasibility-aware ordering. The mutant is box-clipped *before* binomial crossover. For $j = 1, \dots, D$,

$$u_{i,j} = \begin{cases} \mathcal{B}(\mathbf{v}_i)_j, & r_j < CR \text{ or } j = j_{\text{rand}}, \\ q_{i,j}, & \text{otherwise,} \end{cases} \quad (9)$$

with independent $r_j \sim U(0, 1)$ and a uniformly chosen forced-crossover dimension j_{rand} . The trial is box-clipped again, evaluated under repair mode, and accepted if the feasibility-aware comparison rule judges its evaluation record no worse than the incumbent. On acceptance the algorithm stores the *box-clipped trial* $\mathbf{q}_i \leftarrow \mathcal{B}(\mathbf{u}_i)$ and its corresponding repaired evaluation record. On rejection it retains both incumbent components. The code does not overwrite population state with the fully repaired point.

The ablation **CREO-DE-NoRG** uses the same DE, repair, repaired-best attraction, noise, crossover and selection mechanisms, changing only β from 0.90 to 0. In contrast, the conventional DE comparator uses the same $F = 0.7$ and $CR = 0.9$ but penalty-mode evaluation, so it is *not* a controlled β -only comparison.

3.5. Implementation-Level Algorithm

Algorithm 1 CREO-DE as implemented in the CEC2017 scripts

1. Initialise $\mathbf{q}_i \sim U([-100, 100]^D)$ for $i = 1, \dots, N$.
 2. Build repair-mode evaluation records for all $\mathbf{q}_i$.
 3. Set the best-so-far record under feasibility-aware ordering.
 4. **while** remaining objective-evaluation budget > 0 **do**
 5. Set $\mathbf{z}_{best}$ to the best-so-far record's repaired point.
 6. **for** each target i , until the budget is exhausted **do**
 7. Read $\mathbf{d}_i$ from the target's stored record.
 8. Sample r_1, r_2, r_3 and $\boldsymbol{\epsilon}_i$.
 9. Form and clip the mutant using Eq. (8).
 10. Binomially cross with stored $\mathbf{q}_i$; clip the trial.
 11. Evaluate the trial using the iterative repair routine.
 12. Increment the CEC objective-evaluation counter.
 13. **if** the trial is no worse under feasibility-aware comparison **then**
 14. Store the clipped trial and its repaired evaluation record.
 15. **end if**
 16. **end for**
 17. Update the best-so-far evaluation record.
 18. **end while**
 19. Return the best-so-far repaired evaluation record.
-

3.6. Computational Cost

Clipping, auxiliary-violation calculation, shrinkage repair (at most ten passes), displacement, and vector arithmetic each require $\mathcal{O}(D)$ operations per candidate. The implemented repair therefore adds at most a constant number of linear passes to the DE vector-processing work. This bound does not include the CEC objective-evaluation cost; the experiments do not report wall-clock overhead measurements.

4. Experimental Methodology

4.1. Research Questions

The experiments are organised around three questions:

- **RQ1:** How does CREO-DE compare with established population-based baselines across the CEC2017 benchmark?
- **RQ2:** Does the repair-guidance term contribute measurable performance changes when the rest of the CREO-DE hybrid is held fixed?
- **RQ3:** Does the usefulness of repair guidance change as the problem dimension increases from $D = 10$ to $D = 30$?

4.2. CEC2017 Benchmark

The evaluation uses the CEC2017 single-objective real-parameter objective functions [13] inside the auxiliary repair/penalty protocol defined in Section 3. The suite contains unimodal, multimodal, hybrid, and composition functions designed to expose different search difficulties. Following the benchmark implementation used in the experiment, F2 is excluded, leaving 29 functions.

CEC functions have different offsets and objective-value scales. Accordingly, raw function values are reported function-by-function, while cross-function conclusions use ranks rather than arithmetic averages of heterogeneous objective values. This follows common recommendations for multi-problem comparison of stochastic optimisers [34, 35].

4.3. Experiment 1: Main Benchmark Comparison

The principal comparison involves Genetic Algorithm (GA), Particle Swarm Optimisation (PSO), Differential Evolution (DE), Grey Wolf Optimiser (GWO), and CREO-DE. The archived main-experiment scripts also run two related variants, CREO-Repair and CREO-RG; the five-method main table focuses on the four reference methods and CREO-DE, while the archived CSV files retain all seven. Each method starts from a uniformly sampled population in $[-100, 100]^D$. GA uses three-way tournament selection, one-point crossover and Gaussian mutation with scale 0.05 times the coordinate range. PSO uses inertia $w = 0.7$, cognitive/social gains $c_1 = c_2 = 1.5$, and a velocity cap of 0.15 times the coordinate range. DE uses $F = 0.7$, $CR = 0.9$ and binomial crossover; GWO uses a linearly decreasing control factor. The CREO-DE parameters and feasibility modes are specified in Section 3.

The scripts specify $D \in \{10, 30\}$, population size $N = 50$, 30 successive pseudorandom runs per function and algorithm, and $10,000D$ calls to the CEC objective per run. F2 is excluded, giving 29 functions per dimension. Hence the main experiment contains $29 \times 30 = 870$ completed run outcomes *per algorithm per dimension*. The scripts initialise Python's and NumPy's global pseudorandom generators with seed 42 once at program startup and then advance the streams across algorithms, runs and functions; they do not reseed each run with a common paired seed. This matters when interpreting run-index-based statistical tests.

All algorithms call the same CEC objective evaluator but do not solve identical *unmodified* CEC2017 selection problems: GA, PSO, DE and GWO use penalty-mode evaluation, whereas the three CREO variants evaluate repaired points. The auxiliary preference $V = 0$ is common as a feasibility criterion, but its handling and the evaluated point differ by method. The reported comparison therefore characterises the performance of these implemented search protocols; it cannot isolate the effect of adding the CREO update to canonical unconstrained CEC2017 benchmarking.

4.4. Evaluation Measures

For each function, the manuscript reports the arithmetic mean of the stored **objective** values over 30 runs. These are raw CEC objective values at the mode-specific evaluated point, not the penalised **fitness** used internally for selection. Lower values are better within a given function and evaluation mode, but the main comparison combines distinct selection/evaluation protocols. To summarise performance across functions, the algorithms are ranked separately on each function. The cross-function table uses mean-score ranks with midranks for exact ties; the archived scripts also produce separate run-level rank summaries

whose meaning should not be conflated with the manuscript’s function-wise mean-score ranks. The following aggregate descriptors are then reported:

- mean function-wise rank;
- standard deviation of function-wise rank;
- number of functions on which the method attains the lowest unrounded archived mean (best-count);
- number of functions on which it attains the highest unrounded archived mean (worst-count); and
- ordinal position obtained by sorting mean ranks.

These rank summaries are descriptive comparisons rather than claims of inferential significance. The main experiment exports unadjusted two-sided Wilcoxon tests between the 30 CREO-DE run-indexed outcomes and those of each comparator, but its single startup seed and sequential random streams do not establish a matched-pair design, and the tests are not corrected for multiple comparisons. Accordingly, no significance conclusions are inferred from those exported p -values here. Properly documented independent run seeds and corrected multi-problem tests are needed for stronger inferential statements [34, 35].

4.5. Experiment 2: Repair-Guidance Ablation

The ablation directly compares the full CREO-DE hybrid with CREO-DE-NoRG using the same repaired-point evaluation, population representation, F , CR , α , γ , and feasibility-aware selection. Only the repair-displacement weight changes, from $\beta = 0.90$ to $\beta = 0$. A third comparator, DE, retains penalty-mode evaluation and is therefore a contextual benchmark rather than an otherwise matched ablation.

Both dimensional result folders contain 29 functions and three algorithm summaries per function. The repository’s present `ablation_experiment.py` configuration has `DIMENSIONS=[30]`; reproducing the archived $D = 10$ result requires changing it to `[10]` and running the script separately, with a separate output folder. The published ablation win/tie/loss counts compare the archived function-wise means, not the outcome of a new execution performed during manuscript revision.

4.6. Implementation and Reproducibility Artifacts

The implementation archive is organised so that the main benchmark and the ablation can be reproduced independently. The accompanying repository contains the core CREO implementation, a CEC2017 benchmark implementation, separate dimension-specific experiment drivers, an ablation driver, dependency information, and result archives. Table 1 summarises the artefacts visible in the repository used for this study.

The archived project and its new analysis package include numerical summaries and per-function convergence plots, but the original ZIP does not contain the required `cec17_python-master.zip`, the full run-level objective arrays, or the underlying independently verifiable version of the CEC evaluator. `CREO.py` additionally requires `pretty_midi` for the separate music demonstration. The inspected source snapshot corresponds to commit `5e04fb2`; the documentation improvements accompanying this revision were prepared separately and are not claimed to be present in that earlier commit.

5. Results

5.1. Aggregate Rank Performance

Table 2 summarises Experiment 1 under the augmented feasibility/repair protocol. The penalty-mode DE run has the lowest mean rank at both dimensions, followed by PSO. These ranks are descriptive for the implemented protocol, rather than a controlled comparison on identical feasible search domains and candidate-evaluation mappings. CREO-DE is third at $D = 10$ but falls to fourth at $D = 30$. The best-count and worst-count columns make the same trend visible from another perspective: CREO-DE attains the lowest unrounded mean on three $D = 10$ functions and on none of the $D = 30$ functions.

At $D = 10$, CREO-DE obtains the lowest unrounded mean on F21, F22, and F25. On F3 and F9 the values appear tied after rounding for display, but the archived unrounded results favour DE (and PSO on F3). At $D = 30$, however, CREO-DE no longer achieves a best-count and is overtaken by GA in aggregate rank. This is the first indication that the fixed repair-guidance formulation is sensitive to dimension.

Table 1

Repository structure supporting the reported experiments.

Repository artefact	Role in reproducibility
CREO.py	Separate music-oriented CREO implementation; not the CEC2017 driver used for the numerical tables.
CREO-CEC2017.py	Alternative CEC2017 driver; actual main reported D10/D30 runs have separate scripts and require an external CEC2017 package.
experiment_d10.py, experiment_d30.py	Dimension-specific drivers for the main benchmark comparison.
ablation_experiment.py	Driver for the repair-guidance ablation.
cec17_results_D10, cec17_results_D30	Archived outputs for the main $D = 10$ and $D = 30$ experiments.
cec17_ablation_small_D10, cec17_ablation_small_D30	Archived outputs for the corresponding ablation experiments.
requirements.txt	Corrected pip-compatible list of the Python dependencies required by the original scripts; historical dependency versions were not recorded.
results.txt	Repository-level results summary and result pointer.

Table 2

Experiment 1 aggregate rank statistics derived from function-wise mean objective values. Lower rank is better; best/worst counts use unrounded archived means and include exact ties only.

Algorithm	$D = 10$					$D = 30$				
	Mean rank	Rank SD	Best	Worst	Pos.	Mean rank	Rank SD	Best	Worst	Pos.
DE	1.190	0.388	24	0	1	1.379	0.677	20	0	1
PSO	2.603	0.958	3	0	2	1.862	0.789	9	0	2
CREO-DE	2.862	1.093	3	2	3	3.517	0.738	0	1	4
GA	3.655	0.857	0	5	4	3.483	0.738	0	4	3
GWO	4.690	0.604	0	22	5	4.759	0.636	0	24	5

Table 3

Supplementary analysis of Experiment 1: all seven archived algorithms, ranked within each CEC2017 function using unrounded mean objective values. “Best” and “worst” include exact ties; numerical ties can make counts differ from 29. Main and ablation archives are separate experiments.

Method	$D = 10$				$D = 30$			
	Mean rank	Rank SD	Best	Worst	Mean rank	Rank SD	Best	Worst
DE	1.190	0.388	24	0	1.483	1.022	20	0
PSO	2.638	1.026	3	0	1.931	0.884	9	0
CREO-DE	3.069	1.361	3	1	4.759	1.380	0	1
GA	4.034	1.476	0	4	4.690	1.312	0	3
GWO	5.379	0.979	0	5	6.138	1.060	0	16
CREO-Repair	5.621	1.083	0	5	4.310	1.671	0	4
CREO-RG	6.069	1.223	0	14	4.690	1.491	0	5

5.2. Full Seven-Method Archive Analysis

The main experiment archived two additional CREO variants beyond the five methods in Table 2. Table 3 reanalyses all seven algorithms using the full-precision archived mean objective values. At $D = 10$, CREO-DE has mean rank 3.069 among all seven methods; at $D = 30$ it has mean rank 4.759. The CREO-Repair and CREO-RG variants do not dominate the main comparison across the 29 functions. The complete seven-method function-wise values for both dimensions are provided in Appendix A. These results are newly derived tabulations of existing CSV summaries, not additional optimiser executions.

5.3. Function-Wise Interpretation

The complete function-wise tables show why a single aggregate statement would be misleading. At $D = 10$, DE dominates many functions, particularly those on which it reaches or approaches the benchmark offset, but CREO-DE remains competitive on several later-index functions and obtains the lowest mean on F21, F22, and F25. On F3 and F9 it approaches, but does not equal, the lowest unrounded mean. Conversely, on functions such as F1, F12, F18, and F30, the gap to DE is large.

At $D = 30$, the picture changes substantially. DE or PSO achieves the lowest mean on every listed function. CREO-DE remains numerically competitive on selected cases, but the systematic loss of best-count and the decline in average rank indicate that the repair-guided term is not robust to the dimensional increase in its present form.

The function-wise evidence therefore supports a conditional interpretation: repair guidance can alter search beneficially on selected lower-dimensional landscapes, but the effect is not uniform and becomes predominantly adverse at the higher tested dimension.

5.4. Dimensional Shift in Experiment 1

The change from $D = 10$ to $D = 30$ is visible in several aggregate descriptors. CREO-DE's mean rank moves from 2.862 to 3.517, its best-count falls from three to zero, and GA overtakes it in aggregate position. By contrast, PSO's best-count increases from three to nine and its mean rank improves from 2.621 to 1.862. The DE comparator remains first in both settings, although its best-count decreases from 24 to 20 because PSO becomes more competitive on the higher-dimensional archive.

The function-level changes are also informative. At $D = 10$, CREO-DE reaches the lowest reported mean on F21, F22, and F25 and approaches the lowest mean on F3 and F9 without an exact tie. At $D = 30$, these advantages disappear: PSO is best on F3, F5, F7, F8, F10, F12, F16, F21, and F28, while DE is best on the remaining reported functions. On several higher-dimensional functions—including F1, F13, F15, F18, F19, and F30—the absolute gap between CREO-DE and the best baseline is large. The deterioration is therefore not caused by a single outlier function; it is distributed across multiple parts of the benchmark.

This pattern is important for interpreting the ablation. If repair guidance were merely neutral at higher dimension, one would expect the full and non-guided variants to remain broadly comparable. The near-uniform ablation losses at $D = 30$ instead indicate that the fixed guidance term becomes systematically harmful under the tested scaling.

5.5. Repair-Guidance Ablation

Table 6 reports Experiment 2. The comparison against CREO-DE-NoRG is the most direct test of the proposed mechanism because the ablated algorithm disables repair guidance by setting $\beta = 0$.

At $D = 10$, CREO-DE wins 12 of the 29 function-wise comparisons against the non-guided variant and loses 17. The result is not a majority win, but it demonstrates that the repair-displacement term is behaviourally consequential and beneficial on a substantial subset of functions. At $D = 30$, the outcome changes to 1/0/28, showing that the same fixed guidance mechanism is almost uniformly detrimental relative to the matched non-guided hybrid.

The ablation rank summaries in Table 7 are consistent with the win/tie/loss evidence and are computed only within the ablation archive. They must not be mixed with the main experiment's separate DE and CREO-DE runs. Full function-wise ablation means appear in Appendix A.

The comparison with the implemented DE comparator is even more restrictive: CREO-DE wins only two functions at $D = 10$ and none at $D = 30$. These results reinforce the central positioning of the paper. The current contribution is not a new state-of-the-art benchmark winner; it is a mechanism whose effect can be measured, isolated, and studied.

6. Discussion

6.1. What the Experiments Establish

Taken together, the two experiments establish three main points. First, repair displacement is not merely bookkeeping within this implementation: changing β changes the optimisation outcome across many functions. Second, the directionality is problem-dependent at $D = 10$, where the guided and non-guided

Table 4

Experiment 1: function-wise CEC2017 performance for $D = 10$. Values are mean objective values over 30 stochastic runs; lower is better. Bold denotes the lowest unrounded archived mean among the five principal methods in each row; selected near-ties are displayed with extra precision.

Function	GA	PSO	DE	GWO	CREO-DE
F1	1.20e+08	754.46	100.00	6.57e+08	1.57e+06
F3	836.17	300.00	300.00	4832.87	300.002
F4	414.87	400.89	400.00	459.48	402.00
F5	532.09	527.00	511.47	540.65	533.67
F6	608.92	601.01	600.00	609.38	601.92
F7	756.13	721.33	732.37	751.31	744.22
F8	828.13	816.95	816.73	826.38	834.14
F9	935.47	900.14	900.00	933.66	900.00018
F10	1919.18	1663.21	1698.88	2167.25	2173.32
F11	1145.77	1118.66	1101.03	1189.35	1121.52
F12	2.97e+06	1.08e+04	1206.02	2.12e+06	1.04e+05
F13	1.28e+04	9430.45	1300.93	8615.48	4087.35
F14	1494.44	1451.69	1400.93	3417.49	1615.04
F15	1869.96	1554.75	1500.47	7985.17	2193.77
F16	1672.26	1875.13	1616.03	1899.10	1649.85
F17	1757.92	1757.16	1701.65	1774.22	1758.66
F18	2.27e+04	4503.45	1800.22	1.00e+04	9312.51
F19	2112.11	1982.14	1900.02	1.83e+04	2024.33
F20	2066.89	2093.19	2000.40	2161.54	2076.18
F21	2310.10	2304.89	2253.27	2330.23	2241.93
F22	2323.58	2301.84	2301.46	2336.40	2292.39
F23	2642.93	2646.11	2608.22	2670.57	2646.28
F24	2689.37	2678.54	2552.30	2764.53	2696.10
F25	2929.32	2927.05	2902.37	2935.53	2900.81
F26	2961.98	2939.39	2900.00	3048.17	2903.18
F27	3124.97	3151.27	3096.19	3171.77	3116.27
F28	3155.83	3181.48	3100.00	3394.92	3105.72
F29	3206.28	3215.71	3135.92	3260.95	3189.98
F30	2.19e+05	3.55e+05	3648.00	1.20e+06	1.79e+04

variants alternate in their advantages. Third, the current fixed repair-guidance formulation does not scale reliably from $D = 10$ to $D = 30$.

This distinction matters. A method can contain a meaningful search signal without that signal being well calibrated. The ablation shows that repair displacement contains information capable of changing outcomes; the dimensional result shows that a constant weighting of that information is not sufficient for robust performance.

Table 5

Experiment 1: function-wise CEC2017 performance for $D = 30$. Values are mean objective values over 30 stochastic runs; lower is better. Bold denotes the lowest unrounded archived mean among the five principal methods in each row; selected near-ties are displayed with extra precision.

Function	GA	PSO	DE	GWO	CREO-DE
F1	2.77e+09	2654.62	100.00	1.32e+10	1.22e+09
F3	2.24e+04	300.00	1072.08	7.61e+04	5.01e+04
F4	1031.63	454.33	415.17	2652.20	1149.87
F5	732.90	644.19	654.62	791.17	745.19
F6	632.78	630.02	600.00	656.75	631.75
F7	1053.84	831.40	912.17	1023.45	1004.33
F8	1020.47	934.05	953.32	1022.75	1039.34
F9	2388.13	2978.41	900.11	4011.73	2089.95
F10	8012.23	4596.73	7928.52	6185.94	7861.13
F11	1853.22	1226.20	1138.52	4685.94	2032.38
F12	3.59e+08	1.70e+04	1.86e+04	1.44e+09	6.12e+08
F13	1.14e+08	1.13e+04	1403.04	7.01e+08	1.60e+08
F14	7.00e+04	4716.14	1465.59	8.99e+05	6.30e+04
F15	1.47e+07	3720.50	1526.51	4.23e+06	2.83e+06
F16	3479.76	2621.63	2646.28	3975.41	3540.28
F17	2242.11	2239.83	1802.91	2649.90	2377.87
F18	1.32e+06	6.54e+04	1.08e+04	2.61e+06	5.68e+05
F19	2.12e+07	8079.74	1917.11	1.00e+07	9.59e+06
F20	2582.16	2486.87	2166.06	2663.76	2607.39
F21	2528.59	2427.51	2464.01	2565.84	2549.52
F22	2792.38	3231.73	2300.00	4048.86	2558.28
F23	3009.63	2956.41	2707.86	3383.98	3020.69
F24	3132.40	3060.32	2823.54	3554.22	3169.47
F25	3099.32	2908.36	2899.36	3228.67	3033.18
F26	4749.93	4411.22	2955.26	6528.39	5555.20
F27	3541.18	3470.11	3235.20	4154.72	3442.03
F28	3553.51	3106.89	3127.54	4228.84	3575.45
F29	4304.92	3882.50	3447.37	5015.40	4384.36
F30	2.25e+07	8136.69	6592.33	8.10e+07	3.71e+07

6.2. Why Dimensionality May Matter

The present benchmark establishes the dimensional effect empirically but does not prove its cause. One plausible explanation is that repair displacement becomes progressively less aligned with useful objective directions as more coordinates contribute independent boundary corrections. In low dimension, a repaired displacement can provide a relatively coherent inward direction. In higher dimension, the vector may combine many unrelated corrections and therefore behave more like structured noise.

This interpretation should be treated as a hypothesis for future testing rather than as a conclusion from the current data. A direct next step is to measure alignment between the repair vector and subsequent

Table 6

Experiment 2: function-wise win/tie/loss counts for the CREO-DE ablation on 29 CEC2017 functions; lower objective is better.

Comparison	Wins	Ties	Losses
<i>D</i> = 10			
CREO-DE vs. CREO-DE-NoRG	12	0	17
CREO-DE vs. DE	2	0	27
CREO-DE-NoRG vs. DE	1	0	28
<i>D</i> = 30			
CREO-DE vs. CREO-DE-NoRG	1	0	28
CREO-DE vs. DE	0	0	29
CREO-DE-NoRG vs. DE	1	0	28

Table 7

Experiment 2: three-method ablation archive ranks across 29 functions, computed separately for each dimension from unrounded archived function-wise means.

Method	<i>D</i> = 10			<i>D</i> = 30		
	Mean rank	Best	Worst	Mean rank	Best	Worst
DE	1.103	27	1	1.034	28	0
CREO-DE-NoRG	2.379	1	12	2.000	1	1
CREO-DE	2.517	1	16	2.966	0	28

successful search directions, for example through cosine similarity, and to test dimension-dependent weighting schedules such as $\beta(D)$.

6.3. Relation to Differential Evolution

The implemented DE comparator remains the strongest aggregate method in the current benchmark. This is consistent with the broader literature showing the effectiveness of differential mutation and the importance of careful adaptation of DE control mechanisms [17–20]. CREO-DE should therefore be interpreted as a research vehicle for studying repair-derived information rather than as a replacement for DE.

This positioning also suggests several constructive hybrid directions. Repair guidance could be activated only when a candidate is actually repaired, scaled by repair magnitude, normalised by dimension, conditioned on recent success, or integrated into an adaptive strategy-selection framework. Such modifications would preserve the core CREO idea while reducing the risk that a fixed displacement term dominates useful differential information.

6.4. Reproducibility and Reporting

The repository structure separates the core algorithm, CEC2017 implementation, dimension-specific experiment drivers, ablation driver, environment specification, and result archives. This is important for a mechanism paper because the main claim depends on an implementation-level distinction between the full and ablated algorithms.

The present analysis intentionally uses rank-based summaries for cross-function comparison. Raw CEC objective values span very different numerical scales, so averaging them directly across functions would allow a small number of high-magnitude functions to dominate the summary. Rank-based analysis avoids that scale problem, although it also discards information about the magnitude of performance differences. A complete statistical extension should therefore retain run-level distributions and combine ranks, effect sizes, and corrected nonparametric tests [34, 35].

6.5. Limitations and Threats to Validity

The present evidence is bounded by several design choices. The objective evaluator implements the CEC2017 functions, but the additional preferred-region violation score in Eq. (3) is an author-defined condition. It pulls repaired candidates toward the centre and may bias the outcome on shifted and rotated objective landscapes; this is not a standard unconstrained CEC2017 evaluation. Moreover, penalty-mode and repair-mode algorithms are not subjected to identical evaluation/selection maps, so baseline differences cannot be attributed exclusively to the CREO update. The matched β -ablation is more informative about the guidance term, but even this comparison needs explicit seed tracking and fresh run-level data for stronger inferential conclusions.

Only $D = 10$ and $D = 30$ are documented in the submitted result archive; the separate generic driver lists $D = 50$ in its configuration, which does not constitute observed $D = 50$ evidence. The auxiliary repair parameters (preferred $[-80, 80]$ interval, mean absolute value threshold 60, shrink factor 0.8, and ten-iteration maximum) were not subjected to a sensitivity study. The per-function plots have a generation/iteration x-axis rather than a common objective-evaluation x-axis and are not pooled into an unvalidated cross-function convergence statistic. Complete per-run score/seed files are absent from the historical repository archive. A separately supplied CEC2017 Python/C archive has subsequently been functionally tested and hashed, but no historical checksum exists to prove it is identical to the benchmark version that generated the archived full-budget results. These limitations warrant a controlled same-mode baseline study, archived seed-specific outcomes, and independent replication before broad performance claims are made.

7. Conclusion and Future Work

This paper introduced Repair-Guided Evolutionary Optimisation (CREO), a framework that retains repair-induced displacement and reuses it as directional information during evolutionary search. The proposed CREO-DE instantiation was evaluated on 29 CEC2017 functions at $D = 10$ and $D = 30$ using 30 stochastic runs per function, a population size of 50, and a common budget of $10,000D$ objective evaluations.

Experiment 1 shows that the implemented penalty-mode DE comparator has the lowest aggregate rank at both dimensions in the augmented repair/penalty protocol. CREO-DE is third among the five principal methods at $D = 10$ and fourth at $D = 30$. Experiment 2 isolates the repair-guidance term: CREO-DE records 12/0/17 wins/ties/losses against the non-guided hybrid at $D = 10$, but only 1/0/28 at $D = 30$. Because the main comparator methods use different feasibility/evaluation modes, these results do not support a claim of universal superiority on the unmodified CEC2017 benchmark. They do support the narrower and more informative conclusion that repair displacement can act as a consequential search signal whose usefulness is strongly dimension-dependent.

Future work will investigate dimension-adaptive repair weighting, sensitivity of α , β , and γ , explicit measurements of repair-vector alignment, $D = 50$ and $D = 100$ evaluations, and benchmark suites with nonlinear constraints where repair contains richer feasibility information. CREO can also be embedded into other evolutionary and swarm frameworks to test whether repair-derived directionality generalises beyond the DE instantiation studied here.

Code and Data Availability

The original code and aggregate results were inspected in the supplied CREO-DE-main.zip, corresponding to source snapshot 5e04fb2. A companion package preserves the original experiment scripts and archived results, adds `analysis/rebuild_archived_results.py` for independent reanalysis of the supplied CSV summaries, and includes isolated unit tests of the original repair functions. The aggregate tables and appendix in this paper were re-derived from archived full-precision CSV means; this is not a rerun of the original numerical-optimisation experiment. The files include `experiment_d10.py`, `experiment_d30.py`, `ablation_experiment.py`, aggregated CSVs and per-function plots. The code expects an external `cec17_python-master.zip` archive containing `cec17_functions.py`, `cec17_test_func.c` and CEC2017 input data. A separately supplied Python/C benchmark archive (SHA-256 recorded in the repository's `validation/BENCHMARK_PROVENANCE.json`) was compiled and tested on six evaluator inputs: F1, F3 and F30 at $D = 10$ and $D = 30$. The original main and ablation drivers also completed four isolated

Table 8

Experiment 1: all seven archived algorithms, function-wise mean CEC objective values at $D = 10$. Reported values are descriptive means across 30 stochastic runs; lower is better within a function.

Function	GA	PSO	DE	GWO	CREO-Repair	CREO-RG	CREO-DE
F1	1.20299e+08	754.457	100	6.57083e+08	3513.93	1.41042e+08	1.56642e+06
F3	836.173	300	300	4832.87	2427.68	3362.32	300.002
F4	414.866	400.886	400	459.475	428.83	461.57	401.999
F5	532.086	526.998	511.469	540.649	551.149	551.805	533.674
F6	608.924	601.008	600	609.378	635.738	635.786	601.918
F7	756.127	721.33	732.372	751.315	753.319	744.574	744.215
F8	828.127	816.947	816.732	826.382	823.781	820.696	834.141
F9	935.466	900.139	900	933.655	1301.9	1153.89	900
F10	1919.18	1663.21	1698.88	2167.25	2266.24	2367.3	2173.32
F11	1145.77	1118.66	1101.03	1189.35	1154.33	1168.95	1121.52
F12	2.96695e+06	10808.1	1206.02	2.11699e+06	45757.8	205953	104189
F13	12833.9	9430.45	1300.93	8615.48	11058.4	9209.08	4087.35
F14	1494.44	1451.69	1400.93	3417.49	3015.23	3989.05	1615.04
F15	1869.96	1554.75	1500.47	7985.17	10333.2	14232.2	2193.77
F16	1672.26	1875.13	1616.03	1899.1	2000.05	2020.82	1649.85
F17	1757.92	1757.16	1701.65	1774.22	1796.2	1804.88	1758.66
F18	22663.4	4503.45	1800.22	10028.7	7522.17	6109.03	9312.51
F19	2112.11	1982.14	1900.02	18276.3	3605.73	4292.93	2024.33
F20	2066.89	2093.19	2000.4	2161.54	2219.33	2195.91	2076.18
F21	2310.1	2304.89	2253.27	2330.23	2313.18	2329.07	2241.93
F22	2323.58	2301.84	2301.46	2336.4	2592	2647.99	2292.39
F23	2642.93	2646.11	2608.22	2670.57	2754.13	2780.86	2646.28
F24	2689.37	2678.54	2552.3	2764.53	2777	2752.27	2696.1
F25	2929.32	2927.05	2902.37	2935.53	2952.8	2939.94	2900.81
F26	2961.98	2939.39	2900	3048.17	3582.35	3509	2903.18
F27	3124.97	3151.27	3096.19	3171.77	3257.54	3266.9	3116.27
F28	3155.83	3181.48	3100	3394.92	3418.89	3456.92	3105.72
F29	3206.28	3215.71	3135.92	3260.95	3332.16	3397.73	3189.98
F30	219122	355358	3648	1.19944e+06	2.14589e+06	4.38457e+06	17915.6

one-function, one-run, reduced-budget smoke experiments. These functional checks do not reproduce the published 29-function, 30-run results, and the benchmark archive used historically cannot be established as byte-identical without a contemporaneous checksum. Complete historical run-level outcomes and a verified public upstream source/license were not provided. The third-party archive is therefore kept as a local dependency rather than included in the public repository package. The packaged repository now includes expanded reproduction instructions, a corrected dependency list, unit tests of the archived repair functions, and a script that reanalyses the archived summary tables. These additions and the reduced-budget verification do not substitute for full-budget independent replication or the missing historical run-level results.

A. Complete Archived Function-Wise Results

The following supplementary tables report the full-precision archived CSV source values to six significant digits for readability. Inferences and ranks use the unrounded values stored in the repository's original `all_benchmark_results.csv` and `all_ablation_results.csv` files. The two experiments were executed separately; their similarly named algorithm rows should not be treated as paired observations across experiments.

Table 9

Experiment 1: all seven archived algorithms, function-wise mean CEC objective values at $D = 30$. Reported values are descriptive means across 30 stochastic runs; lower is better within a function.

Function	GA	PSO	DE	GWO	CREO-Repair	CREO-RG	CREO-DE
F1	2.7686e+09	2654.62	100	1.32197e+10	51696.6	45024.6	1.22351e+09
F3	22416	300	1072.08	76083	2657.19	9892.86	50134.5
F4	1031.63	454.328	415.169	2652.2	540.7	609.46	1149.87
F5	732.897	644.185	654.623	791.171	695.039	712.975	745.191
F6	632.781	630.018	600	656.745	656.202	655.737	631.753
F7	1053.84	831.404	912.175	1023.45	1181.76	1108.44	1004.33
F8	1020.47	934.053	953.322	1022.75	956.364	947.571	1039.34
F9	2388.13	2978.41	900.109	4011.73	4792.95	4596.93	2089.95
F10	8012.23	4596.73	7928.52	6185.94	5537.69	5615.08	7861.13
F11	1853.22	1226.2	1138.52	4685.94	1226.71	1240.96	2032.38
F12	3.59272e+08	17014.2	18557.4	1.44367e+09	716947	2.80372e+06	6.11596e+08
F13	1.1432e+08	11281.7	1403.04	7.01258e+08	46973.5	46124.5	1.60248e+08
F14	70033.5	4716.14	1465.59	898551	2987.12	2994.96	62993
F15	1.46785e+07	3720.5	1526.51	4.23035e+06	31789.3	23690.9	2.82505e+06
F16	3479.76	2621.63	2646.28	3975.41	3091.81	3183.62	3540.28
F17	2242.11	2239.83	1802.91	2649.9	2676.48	2768.36	2377.87
F18	1.31946e+06	65369.1	10779.4	2.60629e+06	71972.6	73472.1	567619
F19	2.11843e+07	8079.74	1917.11	1.00114e+07	69433.5	78247.8	9.58878e+06
F20	2582.16	2486.87	2166.06	2663.76	2828.17	2764.47	2607.39
F21	2528.59	2427.51	2464.01	2565.84	2513.92	2535.93	2549.52
F22	2792.38	3231.73	2300	4048.86	6531.09	6240.86	2558.28
F23	3009.63	2956.41	2707.86	3383.98	3594.19	3634.12	3020.69
F24	3132.4	3060.32	2823.54	3554.22	3579.3	3606.65	3169.47
F25	3099.32	2908.36	2899.36	3228.67	2943.95	2956.42	3033.18
F26	4749.93	4411.22	2955.26	6528.39	7680	7872.56	5555.2
F27	3541.18	3470.11	3235.2	4154.72	4441.51	4507.85	3442.03
F28	3553.51	3106.89	3127.54	4228.84	3261.01	3340.68	3575.45
F29	4304.92	3882.5	3447.37	5015.4	4794.58	4814.58	4384.36
F30	2.2472e+07	8136.69	6592.33	8.10106e+07	237693	321778	3.70553e+07

Table 10

Experiment 2: complete archived function-wise ablation mean objectives for each dimension. The DE comparator uses penalty-mode evaluation; CREO-DE-NoRG and CREO-DE use repaired-point evaluation.

Function	$D = 10$			$D = 30$		
	DE	CREO-DE-NoRG	CREO-DE	DE	CREO-DE-NoRG	CREO-DE
F1	100	1.2907e+07	1.63754e+06	100	3.33834e+08	1.08224e+09
F3	300	305.312	300.002	1003.68	9519.64	51244.4
F4	400	404.087	401.947	429.365	546.307	1109.03
F5	512.94	525.381	535.979	654.149	700.353	751.263
F6	600	601.5	601.905	600	607.109	632.31
F7	731.653	737.796	744.773	911.06	958.749	1001.4
F8	817.047	831.31	835.267	963.801	1006.16	1043.14
F9	900	900	900	900	995.539	2058.56
F10	1656.12	1852.97	2163.44	7741.15	7693.86	7879.01
F11	1100.9	1116.3	1120.69	1133.33	1305.02	1995.19
F12	1203.19	426862	123248	20653.9	3.63704e+07	5.63961e+08
F13	1300.91	3632.27	3856.39	1410.3	1.06259e+07	1.81075e+08
F14	1401.09	1741.83	1626.18	1457.84	15504.6	57630.9
F15	1500.34	2038.69	2185.22	1534.89	281455	2.96534e+06
F16	1612.16	1615.06	1653.45	2567.72	2878.82	3547.98
F17	1701.4	1750.28	1756.25	1830.07	2093.59	2405.07
F18	1800.29	18874.4	6609.6	10127.7	121950	680445
F19	1900.16	2128.15	2087.18	1917.34	911365	6.66272e+06
F20	2000.13	2074.54	2076.38	2214.28	2315.42	2636.24
F21	2256.09	2282.04	2219.6	2442.73	2498.09	2549.49
F22	2301.39	2296.02	2296.65	2300	2577.03	2549.78
F23	2609.36	2629.81	2645.9	2707.14	2862.94	3018.88
F24	2562.88	2698.75	2665.98	2821.88	3021.75	3178.56
F25	2899.66	2908.95	2901.64	2899.47	2922.27	3032.51
F26	2900	2910.41	2903.57	3043.24	5528.48	5978.22
F27	3096.18	3103.32	3115.49	3235.22	3308.07	3447.14
F28	3100	3111.47	3105.35	3120.66	3250.14	3570.48
F29	3136.64	3164.93	3186.99	3446.81	3904.18	4360.64
F30	3647.97	12752	18479.8	6754.01	3.36715e+06	3.70934e+07

CRediT authorship contribution statement

Alan Immanuel Benjamin Vallavaraj: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review and editing, Visualization.

References

- [1] X.-S. Yang, *Engineering Optimization: An Introduction with Metaheuristic Applications*. Hoboken, NJ, USA: Wiley, 2010.
- [2] E.-G. Talbi, *Metaheuristics: From Design to Implementation*. Hoboken, NJ, USA: Wiley, 2009.
- [3] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York, NY, USA: Springer, 2006.
- [4] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [5] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Cambridge, MA, USA: MIT Press, 1992.
- [6] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proc. IEEE Int. Conf. Neural Networks (ICNN)*, 1995, pp. 1942–1948.
- [7] R. Storn and K. Price, “Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces,” *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.
- [8] S. Mirjalili, S. M. Mirjalili, and A. Lewis, “Grey wolf optimizer,” *Advances in Engineering Software*, vol. 69, pp. 46–61, 2014.
- [9] C. A. C. Coello Coello, “Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: A survey of the state of the art,” *Computer Methods in Applied Mechanics and Engineering*, vol. 191, nos. 11–12, pp. 1245–1287, 2002.
- [10] K. Deb, “An efficient constraint handling method for genetic algorithms,” *Computer Methods in Applied Mechanics and Engineering*, vol. 186, nos. 2–4, pp. 311–338, 2000.
- [11] T. P. Runarsson and X. Yao, “Stochastic ranking for constrained evolutionary optimization,” *IEEE Transactions on Evolutionary Computation*, vol. 4, no. 3, pp. 284–294, 2000.
- [12] E. Mezura-Montes and C. A. C. Coello Coello, “Constraint-handling in nature-inspired numerical optimization: Past, present and future,” *Swarm and Evolutionary Computation*, vol. 1, no. 4, pp. 173–194, 2011.
- [13] N. H. Awad, M. Z. Ali, P. N. Suganthan, J. J. Liang, and B. Y. Qu, “Problem definitions and evaluation criteria for the CEC 2017 special session and competition on single objective real-parameter numerical optimization,” Nanyang Technological University, Singapore, Tech. Rep. 201611, 2016.
- [14] C. Blum and A. Roli, “Metaheuristics in combinatorial optimization: Overview and conceptual comparison,” *ACM Computing Surveys*, vol. 35, no. 3, pp. 268–308, 2003.
- [15] T. Bäck, *Evolutionary Algorithms in Theory and Practice*. New York, NY, USA: Oxford University Press, 1996.
- [16] A. E. Eiben and J. E. Smith, *Introduction to Evolutionary Computing*. Berlin, Germany: Springer, 2003.
- [17] S. Das and P. N. Suganthan, “Differential evolution: A survey of the state-of-the-art,” *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 1, pp. 4–31, 2011.
- [18] J. Brest, S. Greiner, B. Bošković, M. Mernik, and V. Žumer, “Self-adapting control parameters in differential evolution: A comparative study on numerical benchmark problems,” *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 6, pp. 646–657, 2006.
- [19] A. K. Qin, V. L. Huang, and P. N. Suganthan, “Differential evolution algorithm with strategy adaptation for global numerical optimization,” *IEEE Transactions on Evolutionary Computation*, vol. 13, no. 2, pp. 398–417, 2009.
- [20] R. Tanabe and A. Fukunaga, “Improving the search performance of SHADE using linear population size reduction,” in *Proc. IEEE Congress on Evolutionary Computation (CEC)*, 2014, pp. 1658–1665.
- [21] N. Hansen and A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies,” *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.
- [22] M. Clerc and J. Kennedy, “The particle swarm: Explosion, stability, and convergence in a multidimensional complex space,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 1, pp. 58–73, 2002.
- [23] R. C. Eberhart and Y. Shi, “Particle swarm optimization: Developments, applications and resources,” in *Proc. Congress on Evolutionary Computation*, 2001, pp. 81–86.
- [24] J. J. Liang, A. K. Qin, P. N. Suganthan, and S. Baskar, “Comprehensive learning particle swarm optimizer for global optimization of multimodal functions,” *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 3, pp. 281–295, 2006.
- [25] M. Dorigo and L. M. Gambardella, “Ant colony system: A cooperative learning approach to the traveling salesman problem,” *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 53–66, 1997.
- [26] D. Karaboga, “An idea based on honey bee swarm for numerical optimization,” Erciyes University, Engineering Faculty, Computer Engineering Department, Kayseri, Turkey, Tech. Rep. TR06, 2005.
- [27] X.-S. Yang, “Firefly algorithms for multimodal optimization,” in *Stochastic Algorithms: Foundations and Applications*, Lecture Notes in Computer Science, vol. 5792. Berlin, Germany: Springer, 2009, pp. 169–178.
- [28] S. Mirjalili and A. Lewis, “The whale optimization algorithm,” *Advances in Engineering Software*, vol. 95, pp. 51–67, 2016.
- [29] A. A. Heidari, S. Mirjalili, H. Faris, I. Aljarah, M. Mafarja, and H. Chen, “Harris hawks optimization: Algorithm and applications,” *Future Generation Computer Systems*, vol. 97, pp. 849–872, 2019.

- [30] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [31] P. Moscato, "On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms," Caltech Concurrent Computation Program, California Institute of Technology, Pasadena, CA, USA, C3P Report 826, 1989.
- [32] Z. Michalewicz and M. Schoenauer, "Evolutionary algorithms for constrained parameter optimization problems," *Evolutionary Computation*, vol. 4, no. 1, pp. 1–32, 1996.
- [33] S. Koziel and Z. Michalewicz, "Evolutionary algorithms, homomorphous mappings, and constrained parameter optimization," *Evolutionary Computation*, vol. 7, no. 1, pp. 19–44, 1999.
- [34] S. García, D. Molina, M. Lozano, and F. Herrera, "A study on the use of non-parametric tests for analyzing the evolutionary algorithms' behaviour: A case study on the CEC'2005 Special Session on Real Parameter Optimization," *Journal of Heuristics*, vol. 15, no. 6, pp. 617–644, 2009.
- [35] J. Derrac, S. García, D. Molina, and F. Herrera, "A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms," *Swarm and Evolutionary Computation*, vol. 1, no. 1, pp. 3–18, 2011.